

# PL/SQL CURSORS

Oracle 10g Express Edition - Department of Computer Science

## 1. What is a Cursor?

In PL/SQL, whenever Oracle executes a SQL statement, it allocates a private memory area to store the statement and its result set. This memory area is called a **CURSOR**. A cursor acts as a pointer or handle to this memory area, allowing your PL/SQL program to retrieve and process the rows returned by a query — one row at a time.

### Key Analogy

Think of a cursor like a bookmark in a result set — it keeps track of which row you are currently processing, and lets you move through the results row by row.

Oracle supports two types of cursors:

- **Implicit Cursors** — automatically created by Oracle for every DML and SELECT INTO statement.
- **Explicit Cursors** — manually defined and controlled by the programmer, used for multi-row queries.

## 2. Implicit Cursors

Oracle automatically creates an implicit cursor every time a SQL DML statement (INSERT, UPDATE, DELETE) or a SELECT INTO statement is executed. The programmer does not need to declare or manage these cursors — Oracle handles everything internally.

### 2.1 Implicit Cursor Attributes

After every DML or SELECT INTO statement, Oracle sets the following cursor attributes. They are accessed using the prefix **SQL%**:

| Attribute    | Data Type | Description                                                          |
|--------------|-----------|----------------------------------------------------------------------|
| SQL%FOUND    | BOOLEAN   | TRUE if the last DML/SELECT affected at least one row                |
| SQL%NOTFOUND | BOOLEAN   | TRUE if the last DML/SELECT affected no rows                         |
| SQL%ROWCOUNT | NUMBER    | Number of rows affected by the last DML statement                    |
| SQL%ISOPEN   | BOOLEAN   | Always FALSE for implicit cursors (Oracle closes them automatically) |

## 2.2 Example — Using Implicit Cursor with UPDATE

```
DECLARE
  v_rows_updated NUMBER;
BEGIN
  UPDATE employees
  SET   salary = salary * 1.10
  WHERE department_id = 10;

  v_rows_updated := SQL%ROWCOUNT;

  IF SQL%FOUND THEN
    DBMS_OUTPUT.PUT_LINE('Rows updated: ' || v_rows_updated);
  ELSE
    DBMS_OUTPUT.PUT_LINE('No employees found in department 10.');
```

```
  END IF;
END;
```

```
/
```

## 2.3 Example — Implicit Cursor with SELECT INTO

SELECT INTO is used to fetch exactly ONE row into a variable. It raises an exception if zero or more than one row is returned.

```
DECLARE
  v_name  employees.first_name%TYPE;
  v_salary employees.salary%TYPE;
BEGIN
  SELECT first_name, salary
  INTO   v_name, v_salary
  FROM   employees
  WHERE  employee_id = 101;

  DBMS_OUTPUT.PUT_LINE('Name: ' || v_name || ', Salary: ' || v_salary);
END;
```

```
/
```

### Important ⚠

SELECT INTO will raise NO\_DATA\_FOUND if no row matches, and TOO\_MANY\_ROWS if more than one row matches. Always use exception handling with SELECT INTO (covered in the next topic).

## 3. Explicit Cursors

When a query returns multiple rows, we must use an explicit cursor. The programmer explicitly declares the cursor, opens it, fetches rows one by one, and closes it when done.

### 3.1 The Four Steps of an Explicit Cursor

| Step | Command | Description                                                  |
|------|---------|--------------------------------------------------------------|
| 1    | DECLARE | Define the cursor with a SELECT query in the DECLARE section |
| 2    | OPEN    | Execute the query, populate the active set in memory         |
| 3    | FETCH   | Retrieve the next row from the active set into variables     |
| 4    | CLOSE   | Release the memory used by the cursor                        |

### 3.2 Syntax

```
-- Step 1: DECLARE the cursor
CURSOR cursor_name IS
    SELECT column1, column2 FROM table_name WHERE condition;

-- Step 2: OPEN the cursor
OPEN cursor_name;

-- Step 3: FETCH a row
FETCH cursor_name INTO variable1, variable2;

-- Step 4: CLOSE the cursor
CLOSE cursor_name;
```

### 3.3 Complete Example — Explicit Cursor with LOOP

This example retrieves all employees in department 20 and prints their names and salaries.

```
DECLARE
    CURSOR emp_cursor IS
        SELECT first_name, salary
        FROM   employees
        WHERE  department_id = 20
        ORDER BY salary DESC;

    v_name  employees.first_name%TYPE;
    v_salary employees.salary%TYPE;
BEGIN
    OPEN emp_cursor;
```

```

LOOP
  FETCH emp_cursor INTO v_name, v_salary;
  EXIT WHEN emp_cursor%NOTFOUND; -- Exit when no more rows

  DBMS_OUTPUT.PUT_LINE('Name: ' || v_name ||
    ' Salary: ' || v_salary);
END LOOP;

CLOSE emp_cursor;
END;
/

```

## 4. Explicit Cursor Attributes

Just like implicit cursors, explicit cursors have four attributes. They are accessed using `cursor_name%` prefix:

| Attribute                         | Returns | Description                                             |
|-----------------------------------|---------|---------------------------------------------------------|
| <code>cursor_name%FOUND</code>    | BOOLEAN | TRUE if the last FETCH returned a row                   |
| <code>cursor_name%NOTFOUND</code> | BOOLEAN | TRUE if the last FETCH returned no row (end of results) |
| <code>cursor_name%ROWCOUNT</code> | NUMBER  | Number of rows fetched so far                           |
| <code>cursor_name%ISOPEN</code>   | BOOLEAN | TRUE if the cursor is currently open                    |

### 4.1 Example Using %ROWCOUNT and %ISOPEN

```

DECLARE
  CURSOR dept_cursor IS
    SELECT department_name FROM departments;
  v_dept departments.department_name%TYPE;
BEGIN
  OPEN dept_cursor;

  IF dept_cursor%ISOPEN THEN
    DBMS_OUTPUT.PUT_LINE('Cursor is open. Fetching rows...');
  END IF;

  LOOP
    FETCH dept_cursor INTO v_dept;
    EXIT WHEN dept_cursor%NOTFOUND;
    DBMS_OUTPUT.PUT_LINE(dept_cursor%ROWCOUNT || '. ' || v_dept);
  END LOOP;

  DBMS_OUTPUT.PUT_LINE('Total departments: ' || dept_cursor%ROWCOUNT);
  CLOSE dept_cursor;
END;
/

```

## 5. Cursor FOR Loop (Recommended Method)

The Cursor FOR Loop is the most elegant and concise way to work with explicit cursors. Oracle automatically handles the OPEN, FETCH, and CLOSE steps — you only need to declare the cursor and write the loop body.

### Why Use Cursor FOR Loop? ☒

It eliminates the risk of forgetting to close a cursor, reduces code length significantly, and is less prone to programming errors. This is the recommended approach for most PL/SQL cursor work.

### 5.1 Syntax

```
FOR record_variable IN cursor_name LOOP
  -- Use record_variable.column_name to access values
  statements;
END LOOP;
```

### 5.2 Example — Cursor FOR Loop

```
DECLARE
  CURSOR emp_cursor IS
    SELECT first_name, salary, department_id
    FROM   employees
    WHERE  salary > 5000;
BEGIN
  FOR emp_rec IN emp_cursor LOOP
    DBMS_OUTPUT.PUT_LINE(
      'Name: ' || emp_rec.first_name ||
      ' Salary: ' || emp_rec.salary ||
      ' Dept: ' || emp_rec.department_id
    );
  END LOOP;
  -- Cursor is automatically closed here
END;
/
```

### 5.3 Inline Query in FOR Loop (No DECLARE Needed)

You can also write the SELECT query directly inside the FOR loop — no DECLARE section needed:

```
BEGIN
  FOR emp_rec IN (SELECT first_name, salary FROM employees
                  WHERE department_id = 30) LOOP
    DBMS_OUTPUT.PUT_LINE(emp_rec.first_name || ' - ' || emp_rec.salary);
  END LOOP;
END;
```

```
| /
```

## 6. Cursors with Parameters

---

Parameterized cursors allow you to pass values to the cursor at the time of opening, making the cursor reusable for different input values.

### 6.1 Syntax

```
CURSOR cursor_name (param_name datatype) IS  
  SELECT ... FROM ... WHERE column = param_name;
```

### 6.2 Example — Parameterized Cursor

```
DECLARE  
  CURSOR emp_by_dept (p_dept_id NUMBER) IS  
    SELECT first_name, salary  
    FROM employees  
    WHERE department_id = p_dept_id;  
  
BEGIN  
  DBMS_OUTPUT.PUT_LINE('--- Department 10 ---');  
  FOR rec IN emp_by_dept(10) LOOP  
    DBMS_OUTPUT.PUT_LINE(rec.first_name || ' : ' || rec.salary);  
  END LOOP;  
  
  DBMS_OUTPUT.PUT_LINE('--- Department 20 ---');  
  FOR rec IN emp_by_dept(20) LOOP  
    DBMS_OUTPUT.PUT_LINE(rec.first_name || ' : ' || rec.salary);  
  END LOOP;  
END;  
/
```

## 7. Cursor FOR UPDATE and WHERE CURRENT OF

---

When you need to update or delete the row that you just fetched with a cursor, you can use the FOR UPDATE clause in the cursor declaration. This locks the rows when the cursor is opened. You can then use WHERE CURRENT OF to refer to the currently fetched row.

```
DECLARE  
  CURSOR sal_cursor IS  
    SELECT employee_id, salary  
    FROM employees  
    WHERE department_id = 50  
    FOR UPDATE;          -- Locks the rows  
BEGIN
```

```

FOR rec IN sal_cursor LOOP
  IF rec.salary < 3000 THEN
    UPDATE employees
    SET   salary = rec.salary + 500
    WHERE CURRENT OF sal_cursor; -- Updates the current row
  END IF;
END LOOP;
COMMIT;
END;
/

```

**Note on Locking** ⚠

FOR UPDATE locks all selected rows when the cursor is opened. Other sessions cannot modify these rows until a COMMIT or ROLLBACK is issued. Use this carefully in multi-user environments.

## 8. Comparison: Implicit vs Explicit Cursors

| Feature     | Implicit Cursor                  | Explicit Cursor           |
|-------------|----------------------------------|---------------------------|
| Declaration | Auto by Oracle                   | Programmer must declare   |
| Use Case    | Single-row SELECT, DML           | Multi-row SELECT          |
| Control     | Fully automatic                  | Manual (OPEN/FETCH/CLOSE) |
| Attributes  | SQL%FOUND, etc.                  | cursor name%FOUND, etc.   |
| Risk        | NO_DATA_FOUND /<br>TOO_MANY_ROWS | Must remember to CLOSE    |
| Best For    | DML row counts, single lookups   | Processing result sets    |

## 9. Common Mistakes & Best Practices

### 9.1 Common Mistakes

- Forgetting to CLOSE the cursor — leads to resource/memory leaks.
- Using EXIT WHEN before the FETCH — always FETCH first, then check %NOTFOUND.
- Not handling NO\_DATA\_FOUND for SELECT INTO statements.
- Opening a cursor that is already open — causes CURSOR\_ALREADY\_OPEN exception.
- Fetching into wrong number of variables — always match SELECT columns to FETCH variables.

### 9.2 Best Practices

- Prefer Cursor FOR Loop over manual OPEN/FETCH/CLOSE — it is cleaner and safer.

- Always use %TYPE for variable declarations to match column data types automatically.
- Use parameterized cursors when the same query is needed for multiple input values.
- Always COMMIT or ROLLBACK after using FOR UPDATE cursors.
- Add comments in code to identify what each cursor does.

## 10. Lab Exercises

---

### Setup i

Use Oracle 10g XE with the default HR schema or create your own STUDENTS, COURSES, and ENROLLMENTS tables for these exercises.

### Exercise 1 — Basic Implicit Cursor

Write a PL/SQL block that updates the salary of all employees in department 10 by 15%. After the update, use SQL%ROWCOUNT to print how many employees were updated. If no employees exist in department 10, print an appropriate message.

### Exercise 2 — Explicit Cursor with LOOP

Write a PL/SQL block using an explicit cursor that fetches all employees earning more than 8000. Print the employee name, salary, and the current value of %ROWCOUNT for each row fetched.

### Exercise 3 — Cursor FOR Loop

Rewrite Exercise 2 using a Cursor FOR Loop. Notice how much shorter and cleaner the code becomes. Print the same output.

### Exercise 4 — Parameterized Cursor

Create a parameterized cursor that accepts a department ID. Using two separate calls, print all employee names for department 10 and then for department 20.

### Exercise 5 — FOR UPDATE Cursor

Write a PL/SQL block using a FOR UPDATE cursor. For every employee with a salary below 4000, give them a raise of 10% using WHERE CURRENT OF. Commit at the end and print the total number of employees updated.

### Topic Summary

1. Implicit cursors are automatic — used for DML and SELECT INTO.
2. Explicit cursors require DECLARE → OPEN → FETCH → CLOSE lifecycle.
3. Cursor FOR Loop is the most concise and recommended approach.
4. Cursor attributes (%FOUND, %NOTFOUND, %ROWCOUNT, %ISOPEN) control flow.
5. Parameterized cursors make code reusable and flexible.
6. FOR UPDATE with WHERE CURRENT OF allows in-place row modification.